

Telco Customer Churn Analysis

Data Analysis using Python: Pandas, NumPy and Matplotlib

Data heading	IBM Telco Customer Churn (7,043 customers, 33 variables)
Subject	Programming for Data Science
Faculty	Prof. _____
Student 1	Rishiraj Saha PRN: 26020845025
Student 2	Ritika Mondol PRN: 26020845026
Institute	Symbiosis Institute of Business Management
Submission date	24 September 2026

Contents

	Section
Chapter 1	Introduction and Data Description
Chapter 2	Data Analysis using Python
	2.1 Pandas: 3 applications with inference
	2.2 NumPy: 3 applications with inference
	2.3 Matplotlib: 3 applications with inference
Chapter 3	Conclusion: Inference and Storytelling

Chapter 1: Introduction and Data Description

1.1 Introduction

Customer churn is when a subscriber ends their relationship with a service provider. It is one of the most expensive problems in telecommunications: winning a new customer typically costs five to seven times more than keeping an existing one, and every churned customer takes their future recurring revenue with them. Knowing *who* churns, *when* and *why* lets a telecom operator target retention spending where it has the most effect.

This project analyses the IBM Telco Customer Churn dataset with the core Python data-science stack. **Pandas** is used for loading, cleaning, grouping and pivoting; **NumPy** for vectorised statistics, correlation and segmentation; and **Matplotlib** for visual storytelling. Each application is shown as an image of the code, followed by its output and an inference.

1.2 Objectives

- Measure the overall churn rate and its revenue impact.
- Identify the service, contract and billing characteristics associated with churn.
- Understand how churn changes over the customer lifecycle (tenure).
- Summarise the stated reasons for churn and recommend retention actions.

1.3 Data Source

Dataset: Telco Customer Churn (IBM Cognos Analytics sample, version 11.1.3+), published on Kaggle: kaggle.com/datasets/yjchang/telco-customer-churn-1113. It is a secondary dataset describing a fictional telecom company in California, USA, with one row per customer. The file used is *DataSet/Telco_customer_churn.xlsx*.

Property	Value
Rows (customers)	7,043 (minimum required: 510)
Columns	33 in raw file (29 after dropping constant / duplicate columns)
Categorical variables	e.g. Gender, Contract, Internet Service, Payment Method, Churn Label
Numerical variables	Tenure Months, Monthly Charges, Total Charges, CLTV, Churn Score, Latitude/Longitude
Target variable	Churn Value (1 = customer left this quarter, 0 = stayed)
Geography	California, USA: 1,129 cities

1.4 Data Dictionary

Variable	Type	Description
CustomerID	ID	Unique customer identifier
City, Zip Code, Latitude, Longitude	Categorical / Numeric	Customer's primary residence
Gender, Senior Citizen, Partner, Dependents	Categorical	Demographic attributes (Yes/No, Male/Female)
Tenure Months	Numeric	Months the customer has been with the company (0-72)
Phone Service, Multiple Lines	Categorical	Phone subscription details
Internet Service	Categorical	DSL, Fiber optic or No internet
Online Security, Online Backup, Device Protection, Tech Support	Categorical	Add-on services (Yes / No / No internet service)
Streaming TV, Streaming Movies	Categorical	Streaming add-ons
Contract	Categorical	Month-to-month, One year, Two year
Paperless Billing, Payment Method	Categorical	Billing preferences
Monthly Charges	Numeric	Current monthly bill (USD)
Total Charges	Numeric	Total billed to date (USD)
Churn Label / Churn Value	Categorical / Binary	Whether the customer left this quarter
Churn Score	Numeric	IBM SPSS model-predicted churn likelihood (0-100)
CLTV	Numeric	Predicted Customer Lifetime Value
Churn Reason	Categorical	Customer's stated reason for leaving

1.5 Tools and Environment

Python 3 with pandas, NumPy and Matplotlib (openpyxl is used to read the Excel file). All code is in *Python_Project/telco_churn_analysis.py*. The report is generated reproducibly by *build_report.py*, which executes the code and captures its real output.

Chapter 2: Data Analysis using Python

2.0 Setup: Importing Libraries and Loading the Data

Code

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib
4 matplotlib.use("Agg")
5 import matplotlib.pyplot as plt
6 from pathlib import Path
7
8 BASE = Path(__file__).resolve().parent
9 DATA = BASE.parent / "DataSet" / "Telco_customer_churn.xlsx"
10 FIG_DIR = BASE / "outputs" / "figures"
11 FIG_DIR.mkdir(parents=True, exist_ok=True)
12
13 df = pd.read_excel(DATA)
14 print("Shape:", df.shape)
15 print(df.dtypes.value_counts().to_string())
```

Output

```
Shape: (7043, 33)
str      23
int64     6
float64   3
object    1
```

Inference: The Excel file loads into a DataFrame of **7,043 rows and 33 columns**, well above the 510-row minimum. 23 columns are text (categorical) and 9 are numeric, so the dataset mixes both variable types required by the brief.

2.1 Pandas: Three Applications

Pandas 1 - Data inspection and cleaning

Code

```
1 # 'Total Charges' is stored as text: blank strings for brand-new customers
2 df["Total Charges"] = pd.to_numeric(df["Total Charges"], errors="coerce")
3 print("Missing values per column (non-zero only):")
4 print(df.isna().sum()[lambda s: s > 0].to_string())
5
6 # New customers (tenure 0) have not been billed yet -> total charges = 0
7 df["Total Charges"] = df["Total Charges"].fillna(0)
8 # Drop columns that carry no analytical information
9 df = df.drop(columns=["Count", "Country", "State", "Lat Long"])
10
11 num_cols = ["Tenure Months", "Monthly Charges", "Total Charges",
12             "CLTV", "Churn Score"]
13 print("\nCleaned shape:", df.shape)
14 print(df[num_cols].describe().round(2).to_string())
15 print("\nOverall churn rate: {:.2%}".format(df["Churn Value"].mean()))
```

Output

Missing values per column (non-zero only):

Total Charges 11
Churn Reason 5174

Cleaned shape: (7043, 29)

	Tenure Months	Monthly Charges	Total Charges	CLTV	Churn Score
count	7043.00	7043.00	7043.00	7043.00	7043.00
mean	32.37	64.76	2279.73	4400.30	58.70
std	24.56	30.09	2266.79	1183.06	21.53
min	0.00	18.25	0.00	2003.00	5.00
25%	9.00	35.50	398.55	3469.00	40.00
50%	29.00	70.35	1394.55	4527.00	61.00
75%	55.00	89.85	3786.60	5380.50	75.00
max	72.00	118.75	8684.80	6500.00	100.00

Overall churn rate: 26.54%

Inference: Only two columns contain missing values. **Total Charges** has 11 blanks, all for customers with 0 months tenure who have not yet been billed, so they are filled with 0 rather than dropped. The 5,174 blanks in **Churn Reason** are structural: retained customers have no churn reason. After cleaning, the average customer has been with the company for **32 months** and pays **\$64.76/month**. The overall churn rate is **26.5%**: roughly one in four customers left.

Pandas 2 - Churn rate by contract, internet service and payment method

Code

```
1 def churn_summary(col):
2     return (df.groupby(col)
3             .agg(Customers=("CustomerID", "count"),
4                 Churned=("Churn Value", "sum"),
5                 Churn_Rate=("Churn Value", "mean"),
6                 Avg_Monthly=("Monthly Charges", "mean"))
7             .sort_values("Churn_Rate", ascending=False)
8             .round(3))
9
10 for col in ["Contract", "Internet Service", "Payment Method"]:
11     print(f"\n--- Churn by {col} ---")
12     print(churn_summary(col).to_string())
```

Output

```
--- Churn by Contract ---
      Customers  Churned  Churn_Rate  Avg_Monthly
Contract
Month-to-month    3875    1655      0.427      66.398
One year          1473     166      0.113      65.049
Two year          1695     48       0.028      60.770

--- Churn by Internet Service ---
      Customers  Churned  Churn_Rate  Avg_Monthly
Internet Service
Fiber optic       3096    1297      0.419      91.500
DSL               2421     459      0.190      58.102
No                1526     113      0.074      21.079

--- Churn by Payment Method ---
      Customers  Churned  Churn_Rate  Avg_Monthly
Payment Method
Electronic check   2365    1071      0.453      76.256
Mailed check       1612     308      0.191      43.917
Bank transfer (automatic) 1544     258      0.167      67.193
Credit card (automatic) 1522     232      0.152      66.512
```

Inference: Contract type is the strongest categorical churn driver. **Month-to-month customers churn at 42.7%**, against 11.3% on one-year and only 2.8% on two-year contracts. Fibre-optic customers churn at 41.9%, more than twice the DSL rate, even though they pay the most (\$91.50 on average). This points to a price/value problem with the premium product. Customers paying by **electronic check churn at 45.3%**, about three times the rate of those on automatic card or bank payments.

Pandas 3 - Tenure bands, pivot table and churn reasons

Code

```
1 bins = [0, 12, 24, 48, 72]
2 labels = ["0-12 m", "13-24 m", "25-48 m", "49-72 m"]
3 df["Tenure Band"] = pd.cut(df["Tenure Months"], bins=bins, labels=labels,
4                             include_lowest=True)
5
6 pivot = pd.pivot_table(df, values="Churn Value", index="Tenure Band",
7                           columns="Contract", aggfunc="mean", observed=True)
8 print("Churn rate by Tenure Band x Contract:")
9 print((pivot * 100).round(1).to_string())
10
11 print("\nTop 8 churn reasons (share of churned customers):")
12 reasons = (df.loc[df["Churn Value"] == 1, "Churn Reason"]
13             .value_counts(normalize=True))
14 print((reasons.head(8) * 100).round(1).to_string())
```

Output

```
Churn rate by Tenure Band x Contract:
Contract      Month-to-month  One year  Two year
Tenure Band
0-12 m          51.4          10.5       0.0
13-24 m          37.7           8.1       0.0
25-48 m          32.9          10.6       2.2
49-72 m          26.0          12.9       3.3

Top 8 churn reasons (share of churned customers):
Churn Reason
Attitude of support person          10.3
Competitor offered higher download speeds  10.1
Competitor offered more data           8.7
Don't know                           8.2
Competitor made better offer           7.5
Attitude of service provider           7.2
Competitor had better devices           7.0
Network reliability                     5.5
```

Inference: The pivot table shows that contract and tenure add to each other's effect. A month-to-month customer in their first year has a **51.4%** churn rate, while no two-year customer in the first 24 months churned. Even long-standing month-to-month customers (49-72 m) churn at 26%. The reasons list is led by **support-staff attitude (10.3%)**, with competitor offers on speed, data, price and devices close behind. Together, the competitor reasons are the largest group of churn reasons.

2.2 NumPy: Three Applications

NumPy 1 - Descriptive statistics and a Welch t-test on monthly charges

Code

```
1 churned = df.loc[df["Churn Value"] == 1, "Monthly Charges"].to_numpy()
2 stayed = df.loc[df["Churn Value"] == 0, "Monthly Charges"].to_numpy()
3
4 for name, arr in [("Churned", churned), ("Retained", stayed)]:
5     q1, med, q3 = np.percentile(arr, [25, 50, 75])
6     print(f"{name:9s} n={arr.size:5d} mean={np.mean(arr):6.2f} "
7           f"median={med:6.2f} std={np.std(arr, ddof=1):5.2f} IQR=[{q1:.2f}, {q3:.2f}]")
8
9 # Welch's t-statistic computed from first principles with NumPy
10 m1, m2 = churned.mean(), stayed.mean()
11 v1, v2 = churned.var(ddof=1), stayed.var(ddof=1)
12 t_stat = (m1 - m2) / np.sqrt(v1 / churned.size + v2 / stayed.size)
13 print(f"\nDifference in means: {m1 - m2:.2f} USD | Welch t = {t_stat:.2f}")
```

Output

```
Churned    n= 1869  mean= 74.44  median= 79.65  std=24.67  IQR=[56.15, 94.20]
Retained   n= 5174  mean= 61.27  median= 64.43  std=31.09  IQR=[25.10, 88.40]

Difference in means: 13.18 USD | Welch t = 18.41
```

Inference: Churned customers pay **\$13.18 more per month** on average (\$74.44 vs \$61.27), and their median is \$15 higher. Retained customers show a wider spread (std 31.1) because they include many low-cost phone-only plans around \$20. The Welch t-statistic of **18.4** is far above the usual critical value (about 1.96), so the difference in monthly charges is statistically significant: higher bills are associated with more churn.

NumPy 2 - Correlation matrix of numerical drivers

Code

```
1 corr_cols = ["Tenure Months", "Monthly Charges", "Total Charges",
2             "CLTV", "Churn Score", "Churn Value"]
3 X = df[corr_cols].to_numpy(dtype=float)
4 corr = np.corrcoef(X, rowvar=False)
5
6 print(pd.DataFrame(corr, index=corr_cols, columns=corr_cols).round(2).to_string())
7 churn_corr = corr[-1, :-1]
8 order = np.argsort(-np.abs(churn_corr))
9 print("\nStrongest linear relationships with churn:")
10 for i in order:
11     print(f"    {corr_cols[i]:16s} r = {churn_corr[i]:+.3f}")
```

Output

	Tenure Months	Monthly Charges	Total Charges	CLTV	Churn Score	Churn Value
Tenure Months	1.00	0.25	0.83	0.40	-0.22	-0.35
Monthly Charges	0.25	1.00	0.65	0.10	0.13	0.19
Total Charges	0.83	0.65	1.00	0.34	-0.12	-0.20
CLTV	0.40	0.10	0.34	1.00	-0.08	-0.13
Churn Score	-0.22	0.13	-0.12	-0.08	1.00	0.66
Churn Value	-0.35	0.19	-0.20	-0.13	0.66	1.00

Strongest linear relationships with churn:

```
Churn Score      r = +0.665
Tenure Months    r = -0.352
Total Charges    r = -0.198
Monthly Charges  r = +0.193
CLTV             r = -0.127
```

Inference: Among the raw numeric fields, **tenure has the strongest (negative) relationship with churn ($r = -0.35$)**: the longer a customer stays, the less likely they are to leave. Monthly charges are positively related ($r = +0.19$), and CLTV slightly negatively. Tenure and total charges are highly collinear ($r = 0.83$), which is expected because total charges are roughly tenure x monthly charges. IBM's model-based **Churn Score** tracks actual churn well ($r = +0.67$).

NumPy 3 - Vectorised risk segmentation and revenue at risk

Code

```
1 score = df["Churn Score"].to_numpy()
2 monthly = df["Monthly Charges"].to_numpy()
3 actual = df["Churn Value"].to_numpy()
4
5 segment = np.where(score >= 80, "High",
6                     np.where(score >= 50, "Medium", "Low"))
7 print(f"{'Segment':8s}{ 'Customers':>10s}{ 'Actual churn':>14s}{ 'Monthly rev.':>14s}")
8 for seg in ["High", "Medium", "Low"]:
9     mask = segment == seg
10    print(f"{'seg':8s}{mask.sum():10d}{actual[mask].mean():14.1%}"
11          f"{'monthly[mask].sum():14,.0f}")
12
13 lost_monthly = np.sum(monthly * actual)
14 at_risk = np.sum(monthly[(segment == "High") & (actual == 0)])
15 print(f"\nMonthly revenue lost to churn : ${lost_monthly:,.0f} "
16       f"({lost_monthly / monthly.sum():.1%} of total)")
17 print(f"Annualised revenue lost      : ${lost_monthly * 12:,.0f}")
18 print(f"Retained but high-risk revenue: ${at_risk:,.0f} per month")
```

Output

Segment	Customers	Actual churn	Monthly rev.
High	1201	92.0%	88,553
Medium	3309	23.1%	213,166
Low	2533	0.0%	154,398

Monthly revenue lost to churn : \$139,131 (30.5% of total)
 Annualised revenue lost : \$1,669,570
 Retained but high-risk revenue: \$6,060 per month

Inference: Vectorised np.where rules split customers into three risk tiers using the churn score. The **High** tier (score ≥ 80) had a 92% actual churn rate, and the Low tier had none, so the score separates customers well enough to target retention offers. Churned customers took **\$139K of monthly revenue (30.5% of the total), or about \$1.67M a year**. Churned customers pay more than average, so the revenue share (30.5%) is higher than the customer share (26.5%).

2.3 Matplotlib: Three Applications

Matplotlib 1 - Churn overview and churn rate by contract

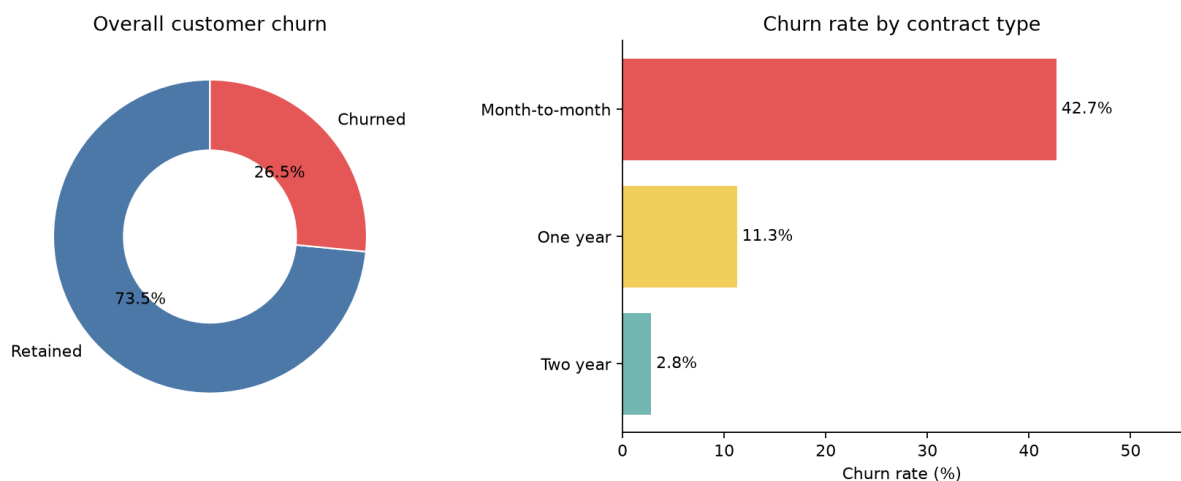
Code

```
1 fig, axes = plt.subplots(1, 2, figsize=(11, 4.3))
2 counts = df["Churn Label"].value_counts()
3 axes[0].pie(counts, labels=["Retained", "Churned"], autopct="%1.1f%%",
4             colors=["#4C78A8", "#E45756"], startangle=90,
5             wedgeprops=dict(width=0.45, edgecolor="white"))
6 axes[0].set_title("Overall customer churn")
7
8 rate = df.groupby("Contract")["Churn Value"].mean().sort_values() * 100
9 bars = axes[1].barh(rate.index, rate.values, color=["#72B7B2", "#F2CF5B", "#E45756"])
10 axes[1].bar_label(bars, fmt="%1.1f%%", padding=3)
11 axes[1].set_xlabel("Churn rate (%)")
12 axes[1].set_title("Churn rate by contract type")
13 axes[1].set_xlim(0, 55)
14 axes[1].spines[["top", "right"]].set_visible(False)
15 fig.tight_layout()
16 fig.savefig(FIG_DIR / "mpl1_churn_contract.png", dpi=160)
17 plt.close(fig)
18 print("Saved figure: mpl1_churn_contract.png")
```

Output

Saved figure: mpl1_churn_contract.png

Result: chart



Inference: The donut chart shows the 26.5% / 73.5% churn split. The bar chart shows how steeply churn changes with contract length: moving a customer from month-to-month to a one-year contract cuts expected churn by about three quarters. Offering incentives to switch to longer contracts is the clearest retention step.

Matplotlib 2 - Distribution of monthly charges and tenure by churn

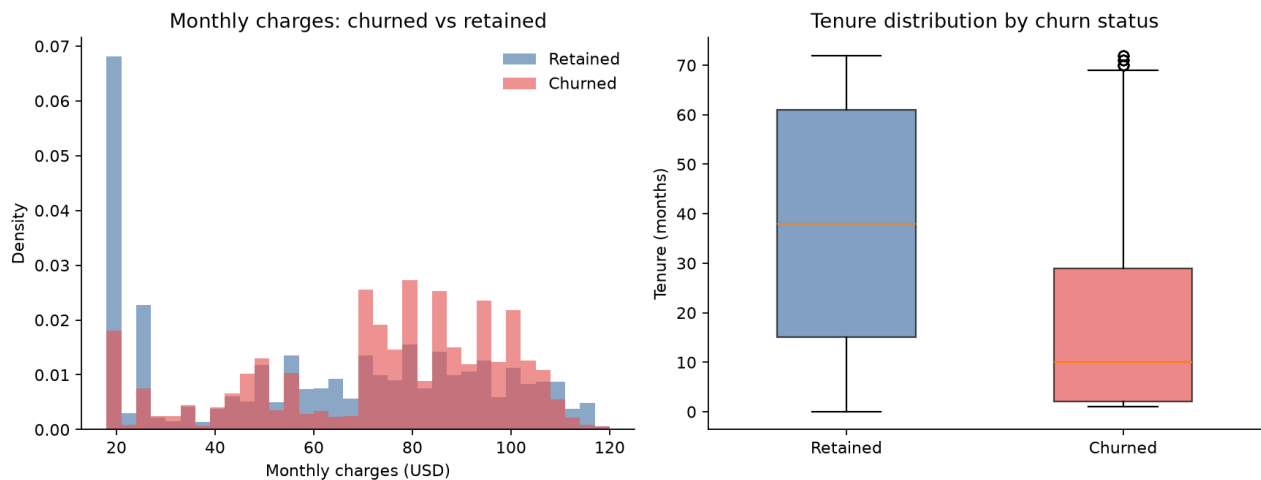
Code

```
1 fig, axes = plt.subplots(1, 2, figsize=(11, 4.3))
2 bins = np.linspace(18, 120, 35)
3 axes[0].hist(stayed, bins=bins, alpha=0.65, label="Retained", color="#4C78A8", density=True)
4 axes[0].hist(churned, bins=bins, alpha=0.65, label="Churned", color="#E45756", density=True)
5 axes[0].set_xlabel("Monthly charges (USD)")
6 axes[0].set_ylabel("Density")
7 axes[0].set_title("Monthly charges: churned vs retained")
8 axes[0].legend(frameon=False)
9
10 groups = [df.loc[df["Churn Value"] == v, "Tenure Months"] for v in (0, 1)]
11 bp = axes[1].boxplot(groups, patch_artist=True, widths=0.5)
12 axes[1].set_xticks([1, 2], ["Retained", "Churned"])
13 for patch, c in zip(bp["boxes"], ["#4C78A8", "#E45756"]):
14     patch.set_facecolor(c); patch.set_alpha(0.7)
15 axes[1].set_ylabel("Tenure (months)")
16 axes[1].set_title("Tenure distribution by churn status")
17 for ax in axes:
18     ax.spines[["top", "right"]].set_visible(False)
19 fig.tight_layout()
20 fig.savefig(FIG_DIR / "mpl2_distributions.png", dpi=160)
21 plt.close(fig)
22 print("Saved figure: mpl2_distributions.png")
```

Output

Saved figure: mpl2_distributions.png

Result: chart



Inference: The histogram has two peaks. Retained customers cluster at **\$20-25** (basic phone plans), while churned customers cluster at **\$70-105**, the fibre/streaming bundle price range. The box plot shows that the median churner leaves at about **10 months**, compared with 38 months tenure for retained customers, so most churn happens early in the customer lifecycle.

Matplotlib 3 - Churn trend over tenure and top churn reasons

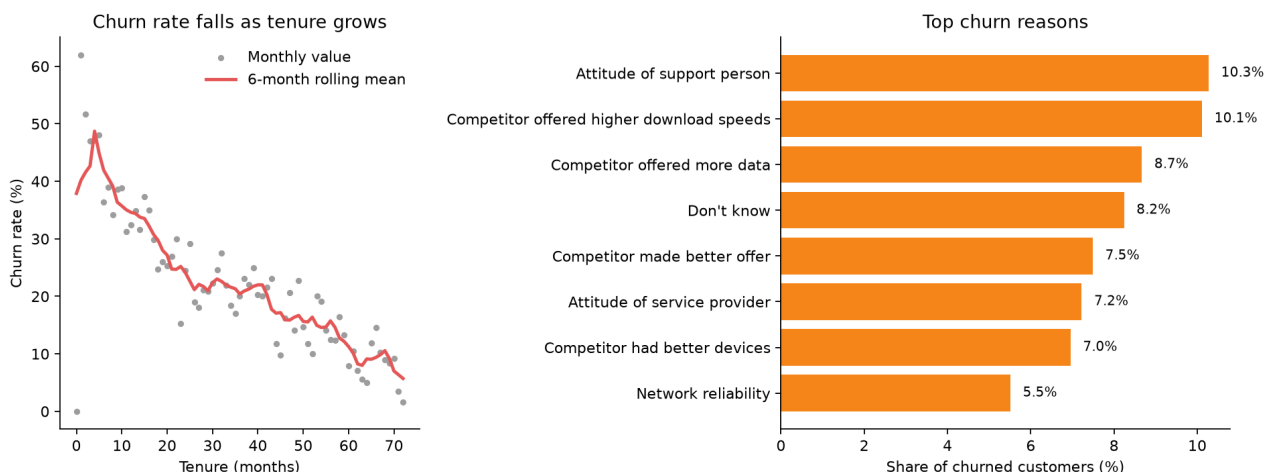
Code

```
1 fig, axes = plt.subplots(1, 2, figsize=(12, 4.6), gridspec_kw={"width_ratios": [1, 1.25]})
2 trend = df.groupby("Tenure Months")["Churn Value"].mean() * 100
3 smooth = trend.rolling(6, center=True, min_periods=1).mean()
4 axes[0].scatter(trend.index, trend.values, s=10, color="#9D9D9D", label="Monthly value")
5 axes[0].plot(smooth.index, smooth.values, color="#E45756", lw=2.2, label="6-month rolling mean")
6 axes[0].set_xlabel("Tenure (months)")
7 axes[0].set_ylabel("Churn rate (%)")
8 axes[0].set_title("Churn rate falls as tenure grows")
9 axes[0].legend(frameon=False)
10
11 top = reasons.head(8).sort_values() * 100
12 axes[1].barh(top.index, top.values, color="#F58518")
13 for y, v in enumerate(top.values):
14     axes[1].text(v + 0.3, y, f"{v:.1f}%", va="center", fontsize=9)
15 axes[1].set_xlabel("Share of churned customers (%)")
16 axes[1].set_title("Top churn reasons")
17 for ax in axes:
18     ax.spines[["top", "right"]].set_visible(False)
19 fig.tight_layout()
20 fig.savefig(FIG_DIR / "mpl3_tenure_reasons.png", dpi=160)
21 plt.close(fig)
22 print("Saved figure: mpl3_tenure_reasons.png")
```

Output

Saved figure: mpl3_tenure_reasons.png

Result: chart



Inference: Churn is highest in the first few months (about 50-60%) and falls steadily to under 10% after five years. This shows that the onboarding period matters most for retention. The reasons chart shows two main themes: **service experience** (support and provider attitude) and **competitive pressure** (speed, data, offers, devices). Retention plans should address both.

Chapter 3: Conclusion: Inference and Storytelling

3.1 The Story the Data Tells

About **one in four customers (26.5%)** left the company, and because churners pay more than average they took **30.5% of monthly revenue** with them, roughly **\$1.67 million a year**. Churn is concentrated in a clear profile rather than spread evenly across customers:

Driver	High-churn group	Low-churn group	Tool / evidence
Contract	Month-to-month: 42.7%	Two-year: 2.8%	Pandas groupby, Matplotlib bar
Tenure	First 12 months: 47.4%	49-72 months: 9.5%	pd.cut, NumPy corr (r = -0.35)
Internet	Fibre optic: 41.9%	No internet: 7.4%	Pandas groupby
Payment	Electronic check: 45.3%	Credit card (auto): 15.2%	Pandas groupby
Price	Churners pay \$74.4 / month	Stayers pay \$61.3 / month	NumPy Welch t = 18.4
Reasons	Competitor offers, support attitude		value_counts, Matplotlib barh

3.2 Key Findings

- **Commitment protects revenue.** Contract length is the single strongest lever. Two-year customers almost never leave, even in their first year.
- **The first year is the danger zone.** Churn peaks in the first few months and falls steadily with tenure. The median churner leaves after about 10 months.
- **The premium product is under-delivering.** Fibre-optic customers pay about \$91/month yet churn at twice the DSL rate. Together with 'competitor offered higher speeds / more data', this suggests a value-for-money gap.
- **Manual payers are less attached.** Electronic-check payers churn about three times more than customers on automatic payments.
- **People matter.** The single most-cited reason is the attitude of support staff, so service quality problems are driving customers away as well as price.
- **Churn risk can be predicted.** A simple NumPy rule on churn score isolates a High-risk tier with 92% actual churn, so retention budgets can be targeted.

3.3 Recommendations

- Offer discounts or loyalty benefits to move month-to-month customers onto 1- or 2-year contracts.
- Run a structured **first-90-days onboarding programme** (welcome calls, set-up help, early-tenure offers).
- Review fibre pricing and speeds against competitors. Bundle Online Security and Tech Support to add value.
- Encourage auto-pay (card / bank transfer) with a small bill credit.
- Invest in support-staff training and track CSAT, as support attitude is the top stated reason for leaving.
- Use the churn-score segmentation to trigger proactive retention calls for High-risk customers.

3.4 Limitations and Future Scope

The data is a fictional, single-quarter snapshot from one US state, so the findings describe associations, not causes. Future work could fit a predictive model (e.g. logistic regression) and estimate the ROI of each retention action.